Especificação Técnica e Governança de Contexto — Módulo TaskAnalyzer

> Bootcamp III — Ciência da Computação · Desafio — Entrega Inicial (Fase 1)
> Autor: Antônio Vinicius Nunes de Alencar · antonio.alencar@sempreceub.com
> Data de elaboração: 11/09/2026 · Versão da especificação: **2.0** (17/10/2026 — Fase 3)

Este documento é a **fonte da verdade** do projeto. O agente de IA implementa o que está aqui — nem mais, nem menos. Qualquer mudança de comportamento exige nova versão deste arquivo, aprovada por humano via Pull Request.

Dois artefatos são **derivados** deste arquivo e nunca editados à mão: `specs/system_design_document.pdf` (versão para leitura e entrega) e `specs/requirements.yaml` (versão legível por máquina do contrato). Em caso de divergência, prevalece este arquivo.

---

1. Visão Geral e Contrato de Negócio (SDD)

1.1 Identificação do estudante

Campo	Valor
Nome completo	Antônio Vinicius Nunes de Alencar
Curso	Ciência da Computação
Campus	Asa Norte
E-mail institucional	antonio.alencar@sempreceub.com
Disciplina	Bootcamp III — 75 horas — EAD (Disciplina Virtual)
Atividade	Desafio — Entrega Inicial (Fase 1 de 3)
Data de elaboração	11/09/2026
Versão da especificação	2.0 (Fase 3 — infraestrutura de CI/CD e containerização)

1.2 Propósito do módulo TaskAnalyzer

O **TaskAnalyzer** é um módulo de análise de tarefas e produtividade. Ele recebe um conjunto de tarefas já registradas por um sistema externo e devolve um relatório consolidado de métricas que apoia a tomada de decisão sobre o desempenho de uma equipe ou de um indivíduo em um período.

O módulo é **puramente analítico e sem estado**: não cria, não edita e não persiste tarefas. Ele apenas recebe dados em memória, valida-os contra o contrato definido neste documento, calcula as métricas e retorna o resultado.

Objetivos funcionais

- Calcular o **tempo médio de conclusão** das tarefas concluídas, em minutos, no total e por prioridade.
- Calcular a **taxa de atraso** percentual (tarefas concluídas após o prazo), no total e por prioridade.
- Contabilizar a **quantidade de tarefas** recebidas, concluídas e em aberto, no total e por prioridade.
- Fornecer **indicadores agrupados por prioridade** (baixa, média, alta) em uma estrutura única e previsível.
- Rejeitar de forma explícita e rastreável qualquer entrada que viole o contrato, em vez de produzir métricas silenciosamente incorretas.

Fora de escopo (por decisão de projeto)

- Persistência em arquivos, bancos de dados ou caches.
- Interface gráfica, API HTTP ou linha de comando.
- Autenticação, autorização ou qualquer controle de acesso.
- Integração com serviços externos, rede ou relógio do sistema para inferir dados ausentes.
- Projeções, previsões ou qualquer inferência estatística além das métricas descritas acima.

1.3 Contrato executável de interface

1.3.1 Assinatura pública

```
def analisar_tarefas(tarefas: list[Tarefa]) -> RelatorioProdutividade:
    """Calcula métricas de produtividade a partir de uma lista de tarefas.

    Args:
        tarefas: Lista de objetos Tarefa previamente validados quanto ao tipo.

    Returns:
        RelatorioProdutividade com as métricas gerais e por prioridade.

    Raises:
        ListaDeTarefasVaziaError: Se a lista recebida estiver vazia.
        ContratoDeEntradaError: Se algum campo obrigatório estiver ausente ou com tipo
incorreto.
        ValorDeDominioInvalidoError: Se prioridade ou status estiverem fora do domínio
permitido.
        DataInconsistenteError: Se houver inconsistência entre as datas de uma tarefa.
        IdentificadorDuplicadoError: Se houver id_tarefa repetido no conjunto.
        SemTarefasConcluidasError: Se nenhuma tarefa do conjunto estiver concluída.
    """
```

`analisar_tarefas` é a **única função pública** do módulo. Funções auxiliares (validação, agrupamento, cálculo de cada métrica) devem existir, mas são privadas, prefixadas com underscore, e não fazem parte do contrato.

1.3.2 Entradas — estrutura `Tarefa`

Campo	Tipo	Obrigatório	Descrição e restrições
`id_tarefa`	`int`	Sim	Identificador único e positivo (> 0). Não pode haver IDs repetidos no mesmo conjunto.

Campo	Tipo	Obrigatório	Descrição e restrições
`titulo`	`str`	Sim	Texto não vazio após remoção de espaços. Máximo de 200 caracteres. Não participa de nenhum cálculo.
`prioridade`	`str`	Sim	Domínio fechado: `"baixa"`, `"media"`, `"alta"`. Comparação sensível a maiúsculas e sem acentuação.
`status`	`str`	Sim	Domínio fechado: `"concluida"`, `"pendente"`, `"cancelada"`.
`data_criacao`	`datetime`	Sim	Momento de criação da tarefa, em UTC (ISO 8601). Referência inicial do cálculo de duração.
`data_inicio`	`datetime | None`	Não	Momento em que a execução começou, em UTC. Quando presente, deve ser ≥ `data_criacao`. Não é usado nos cálculos desta versão.
`prazo`	`datetime`	Sim	Prazo acordado, em UTC. Deve ser > `data_criacao`. Referência para a taxa de atraso.
`data_conclusao`	`datetime | None`	Condicional	Obrigatório quando `status == "concluida"`; deve ser `None` nos demais status. Quando presente, deve ser ≥ `data_criacao`.

1.3.3 Saídas — estrutura `RelatorioProdutividade`

Campo	Tipo	Descrição
`total_tarefas`	`int`	Quantidade total de tarefas recebidas, independentemente do status.
`total_concluidas`	`int`	Quantidade de tarefas com status `"concluida"`.
`total_em_aberto`	`int`	Quantidade de tarefas com status `"pendente"`. Canceladas não entram aqui.
`tempo_medio_conclusao_min`	`float`	Média, em minutos, de (`data_conclusao − data_criacao`) entre as concluídas. Arredondado em 2 casas.
`taxa_atraso_percentual`	`float`	Percentual de concluídas em que `data_conclusao > prazo`, sobre o total de concluídas. Arredondado em 2 casas.
`indicadores_por_prioridade`	`dict[str, IndicadorPrioridade]`	Chaves fixas `"baixa"`, `"media"` e `"alta"`, sempre presentes, mesmo sem tarefas.

Cada valor de `indicadores_por_prioridade` segue a estrutura `IndicadorPrioridade`:

Campo	Tipo	Descrição
`total_tarefas`	`int`	Tarefas daquela prioridade, em qualquer status.
`total_concluidas`	`int`	Tarefas concluídas daquela prioridade.
`tempo_medio_conclusao_min`	`float | None`	`None` quando não há concluídas na prioridade — nunca `0.0`, para não confundir "sem dados" com "instantâneo".
`taxa_atraso_percentual`	`float | None`	`None` quando não há concluídas na prioridade, pela mesma razão.

1.4 Regras de negócio e restrições

1. **RN-01 — Base de cálculo das durações.** Apenas tarefas com status `"concluida"` entram no cálculo de tempo médio e taxa de atraso. Pendentes e canceladas entram apenas nos totais.
2. **RN-02 — Duração da tarefa.** Duração = `data_conclusao - data_criacao`, convertida para minutos como ponto flutuante.
3. **RN-03 — Definição de atraso.** Uma tarefa concluída está atrasada quando `data_conclusao > prazo`. Conclusão exatamente no prazo **não** é atraso.
4. **RN-04 — Fuso horário.** Todas as datas são tratadas em UTC. Datas sem fuso (*naive*) são consideradas UTC; datas com fuso diferente são convertidas para UTC antes de qualquer cálculo.
5. **RN-05 — Arredondamento.** Métricas de ponto flutuante são arredondadas para 2 casas decimais apenas na saída, nunca em cálculos intermediários.
6. **RN-06 — Proteção contra divisão por zero.** Nenhuma divisão sem verificação prévia do denominador. No nível geral, ausência de concluídas é erro (`SemTarefasConcluidasError`); no nível de prioridade, resulta em `None` (seção 1.3.3).
7. **RN-07 — Consistência temporal.** `data_conclusao` e `data_inicio` nunca podem ser anteriores a `data_criacao`; `prazo` deve ser posterior a `data_criacao`.
8. **RN-08 — Coerência entre status e datas.** `"concluida"` exige `data_conclusao` preenchida; `"pendente"` ou `"cancelada"` exigem `data_conclusao` nula.
9. **RN-09 — Unicidade.** IDs duplicados no mesmo conjunto invalidam a análise inteira.
10. **RN-10 — Falha explícita.** Diante de qualquer violação, o módulo interrompe o processamento e levanta a exceção correspondente. Nunca ignora um registro inválido nem substitui valores ausentes por padrões.
11. **RN-11 — Chaves fixas na saída.** As três prioridades sempre aparecem em `indicadores_por_prioridade`, mesmo com zero tarefas.
12. **RN-12 — Imutabilidade da entrada.** O módulo não altera os objetos recebidos. Qualquer normalização (conversão de fuso, por exemplo) ocorre em cópias internas.

1.5 Hierarquia de exceções

Todas as exceções derivam de uma exceção-base própria, de modo que o consumidor possa capturar tudo com um único `except` ou tratar cada caso individualmente.

```
TaskAnalyzerError(Exception)          # base de todas as exceções do módulo
├── ListaDeTarefasVaziaError          # lista recebida sem nenhum elemento
├── ContratoDeEntradaError            # campo obrigatório ausente ou com tipo incorreto
├── ValorDeDominioInvalidoError       # prioridade ou status fora do domínio
├── DataInconsistenteError            # violação das regras RN-07 ou RN-08
├── IdentificadorDuplicadoError       # violação da regra RN-09
└── SemTarefasConcluidasError         # nenhuma tarefa concluída (RN-06)
```

Toda exceção carrega mensagem clara, em português, citando o `id_tarefa` envolvido quando aplicável. Exemplo: `"Tarefa 42: data_conclusao (2026-03-01T08:00Z) anterior à data_criacao (2026-03-02T09:00Z)."`

---

2. Cenários de Aceite e Planejamento do Test Harness

Os cenários estão no formato **Dado – Quando – Então** e descrevem o comportamento esperado, nunca a solução. Eles são **imutáveis**: o assistente de IA não pode alterá-los, relaxá-los ou adaptá-los para fazer o código passar. Se um cenário falhar, o código é que deve mudar.

2.1 Cenário 1 — Sucesso (métricas corretas)

- **Dado** um conjunto de 6 tarefas válidas, com prioridades diferentes, prazos definidos e status variados (concluídas, pendente e cancelada), conforme a massa de dados abaixo;
- **Quando** a função `analisar_tarefas` for executada com esse conjunto;
- **Então** deve retornar um `RelatorioProdutividade` com totais, tempo médio, taxa de atraso e indicadores por prioridade exatamente iguais aos valores esperados.

Massa de dados de referência (ano 2026, todas as datas em UTC)

ID	Título	Prior.	Criação	Prazo	Conclusão	Status
1	Corrigir bug de login	alta	02/03 09:00	02/03 18:00	02/03 12:00	concluida
2	Revisar PR de relatórios	alta	02/03 10:00	02/03 18:00	03/03 10:00	concluida
3	Atualizar documentação da API	media	01/03 08:00	01/03 12:00	01/03 14:00	concluida
4	Refatorar serviço de e-mail	media	03/03 09:00	05/03 18:00	—	pendente
5	Ajustar layout do rodapé	baixa	04/03 11:00	04/03 18:00	04/03 11:30	concluida
6	Migrar banco de testes	alta	27/02 08:00	28/02 18:00	—	cancelada

Memória de cálculo

- Durações das concluídas: tarefa 1 = 180 min; tarefa 2 = 1440 min; tarefa 3 = 360 min; tarefa 5 = 30 min.
- Tempo médio geral = (180 + 1440 + 360 + 30) ÷ 4 = 2010 ÷ 4 = **502,50 min**.
- Atrasadas: tarefa 2 (03/03 10:00 > 02/03 18:00) e tarefa 3 (14:00 > 12:00). Tarefas 1 e 5 dentro do prazo.
- Taxa de atraso geral = 2 ÷ 4 = **50,00%**.
- Tarefa 4 (pendente) e tarefa 6 (cancelada) entram apenas nos totais (RN-01).

Resultado esperado

```
RelatorioProdutividade(
    total_tarefas=6,
    total_concluidas=4,
    total_em_aberto=1,
    tempo_medio_conclusao_min=502.50,
    taxa_atraso_percentual=50.00,
    indicadores_por_prioridade={
        "baixa": IndicadorPrioridade(
            total_tarefas=1, total_concluidas=1,
            tempo_medio_conclusao_min=30.00, taxa_atraso_percentual=0.00,
        ),
        "media": IndicadorPrioridade(
            total_tarefas=2, total_concluidas=1,
            tempo_medio_conclusao_min=360.00, taxa_atraso_percentual=100.00,
        ),
        "alta": IndicadorPrioridade(
            total_tarefas=3, total_concluidas=2,
            tempo_medio_conclusao_min=810.00, taxa_atraso_percentual=50.00,
        ),
    },
)
```

2.2 Cenário 2 — Exceção (entradas inválidas / divisão por zero)

Cinco variações independentes. Cada uma produz uma exceção específica e tratada, com mensagem clara — nunca um resultado parcial, um `None` inesperado ou um `ZeroDivisionError` vazando do Python.

- **Dado** uma entrada que viola o contrato — datas inconsistentes, domínio inválido, identificadores duplicados, lista vazia ou conjunto sem nenhuma tarefa concluída;
- **Quando** a função `analisar_tarefas` for executada com essa entrada;
- **Então** deve disparar a exceção específica correspondente, derivada de `TaskAnalyzerError`, com mensagem objetiva identificando a tarefa e a regra violada, sem produzir nenhum relatório.

Var.	Entrada	Exceção esperada	Regra
2.a	Tarefa concluída com `data_conclusao` anterior à `data_criacao`	`DataInconsistenteError`	RN-07
2.b	Tarefa com `status = "concluida"` e `data_conclusao = None`	`DataInconsistenteError`	RN-08
2.c	Tarefa com `prioridade = "urgente"`	`ValorDeDominioInvalidoError`	Domínio 1.3.2
2.d	Lista vazia (`[]`)	`ListaDeTarefasVaziaError`	RN-06
2.e	Conjunto com 3 tarefas, todas pendentes ou canceladas	`SemTarefasConcluidasError`	RN-06

2.3 Cenários complementares de borda

ID	Situação	Comportamento esperado
3.a	Tarefa concluída exatamente no instante do prazo	Não é atraso (RN-03). Com uma única tarefa nessas condições, taxa de atraso = 0,00%.

ID	Situação	Comportamento esperado
3.b	Prioridade `"baixa"` sem nenhuma tarefa no conjunto	Chave `"baixa"` presente, com totais zerados e métricas `None` (RN-11).
3.c	Todas as tarefas concluídas e atrasadas	Taxa de atraso = 100,00%, sem erro.
3.d	Duas tarefas com o mesmo `id_tarefa`	`IdentificadorDuplicadoError` (RN-09).
3.e	Conclusão no mesmo minuto da criação (duração zero)	Duração 0,0 min é válida e entra normalmente na média.
3.f	Datas fornecidas em fuso diferente de UTC	Convertidas para UTC antes do cálculo; resultado idêntico ao equivalente em UTC (RN-04).

2.4 Planejamento do Test Harness (pytest)

O Test Harness vive em `tests/test_harness.py` e é escrito **antes** da implementação: o código só é aceito quando a suíte inteira passa.

- **Fixtures** — `tarefas_validas()` devolve a massa de dados da seção 2.1; fixtures menores (`tarefa_base()`, `tarefas_sem_concluidas()`) preparam as variações do cenário 2 sem duplicar código.
- **Testes de sucesso** — asserções campo a campo sobre o relatório. Cada métrica tem sua própria asserção, para que a falha aponte diretamente o cálculo quebrado.
- **Testes de exceção** — `pytest.raises` para cada variação, verificando o tipo da exceção e um trecho da mensagem.
- **Parametrização** — `@pytest.mark.parametrize` agrupa as variações do cenário 2 e os cenários de borda.
- **Cobertura** — `pytest --cov=src --cov-report=term-missing`, com meta mínima de **90%** de cobertura de linhas no módulo.

Rastreabilidade cenário → teste

Cenário	Função de teste prevista	Validação
1	`test_metricas_gerais_com_tarefas_validas`	Totais, tempo médio e taxa de atraso gerais
1	`test_indicadores_por_prioridade`	As três chaves e seus quatro campos
2.a / 2.b	`test_datas_inconsistentes_levantam_erro`	Tipo e mensagem de `DataInconsistenteError`
2.c	`test_dominio_invalido_levanta_erro`	`ValorDeDominioInvalidoError`
2.d	`test_lista_vazia_levanta_erro`	`ListaDeTarefasVaziaError`
2.e	`test_sem_concluidas_nao_divide_por_zero`	`SemTarefasConcluidasError`, nunca `ZeroDivisionError`
3.a – 3.f	`test_casos_de_borda` (parametrizado)	Um caso por linha da seção 2.3

---

3. Governança de Contexto

As diretrizes arquiteturais, proibições e regras de interação com a IA estão em `CONTEXT_RULES.md`, na raiz do repositório.

---

4. Arquitetura do Repositório

4.1 Árvore de pastas (estrutura fixa)

```
sdd-task-analyzer/
├── .github/
│   └── workflows/
│       └── ci.yml                    # Pipeline de CI (seção 4.4)
├── Dockerfile                        # Empacotamento (seção 4.5)
├── docker-compose.yml                # Orquestração (seção 4.5)
├── .dockerignore
├── README.md
├── CONTEXT_RULES.md
├── .gitignore
├── requirements.txt
├── specs/
│   ├── task_analyzer_spec.md         # Fonte da verdade
│   ├── system_design_document.pdf    # Derivado deste arquivo
│   └── requirements.yaml             # Derivado deste arquivo
├── tests/
│   └── test_harness.py
└── src/
    └── task_analyzer.py
```

Alterar a árvore é proibido ao agente de IA e exige nova versão desta especificação. A versão 2.0 acrescentou os arquivos de infraestrutura e os dois derivados da especificação; `src/` e `tests/` não mudaram.

4.2 Versionamento

- **Branches:** `main` (estável, recebe apenas releases), `develop` (integração), `feature/*` (um incremento por branch).
- **Merge** somente por Pull Request com revisão humana registrada.
- **Commits** no padrão `tipo: descrição` (`feat`, `fix`, `test`, `docs`, `refactor`, `chore`, `ci`, `build`), com o corpo registrando o que foi gerado por IA e o que foi corrigido manualmente.
- **Proteção de branch:** `main` e `develop` só aceitam merge com o pipeline da seção 4.4 aprovado.
- **Tags:** `v1.0-fase2` ao fim da implementação; `v2.0-fase3` após o pipeline de CI/CD.

4.3 Homologação humana (cinco passos)

1. **Testes automatizados** — suíte completa com cobertura, executada pelo pipeline de CI no Pull Request; qualquer falha reprova.
2. **Conformidade com o contrato** — assinatura, estruturas de entrada/saída, RN-01 a RN-12 e CONTEXT_RULES.
3. **Qualidade de código** — legibilidade, coesão, docstrings, ausência de duplicação e código morto.
4. **Testes manuais de borda** — entradas grandes, combinações incomuns, fusos distintos.
5. **Aprovação e commit** — aprovação no PR, merge e registro do que veio da IA e do que foi corrigido.

Reprovação em qualquer passo devolve o código para nova geração, com o motivo registrado no PR.

4.4 Pipeline de integração contínua (`.github/workflows/ci.yml`)

Item	Contrato
Motor	GitHub Actions
Gatilhos	`push` e `pull_request` nas branches `main` e `develop`
Ambiente	`ubuntu-latest` com Python 3.11
Preparação	`pip install -r requirements.txt` — nenhuma dependência fora desse arquivo
Execução	`pytest tests/test_harness.py -v`, com cobertura de `src/` e meta mínima de 90% (seção 2.4)
Permissões	Somente leitura do repositório (`contents: read`); nenhum segredo é necessário
Aprovação	Execução verde = todos os testes aprovados **e** cobertura ≥ 90%

O harness importa o módulo como `src.task_analyzer`, então a raiz do repositório precisa estar no caminho de importação do Python. O pipeline e a imagem Docker fazem isso definindo a variável de ambiente `PYTHONPATH` na raiz — sem arquivos de configuração do pytest e sem tocar no harness.

4.5 Containerização (`Dockerfile` e `docker-compose.yml`)

Item	Contrato
Imagem base	`python:3.11-slim`
Conteúdo	`requirements.txt`, `src/` e `tests/` — nada além do necessário para rodar a suíte
Usuário	Não-root
Comando de entrada (`CMD`)	`pytest tests/test_harness.py -v`, com os mesmos critérios do pipeline
Orquestração	Serviço `tests` no `docker-compose.yml`; a suíte inteira roda com um único comando: `docker compose run --rm tests`

O contêiner e o pipeline executam exatamente a mesma suíte, com o mesmo critério de aprovação: uma execução local verde no Docker reproduz o resultado do CI em qualquer máquina.

Histórico de revisões

Versão	Data	Alteração
1.0	11/09/2026	Versão original da Fase 1.
1.0 (transposição)	15/10/2026	Transposição para Markdown. Correções de digitação sem mudança de comportamento: (a) "Data de elaboração" na tabela 1.1 preenchida com 11/09/2026; (b) variação 2.a corrigida para "`data_conclusao` anterior à `data_criacao`"; (c) `IdentificadorDuplicadoError` incluída no `Raises` da assinatura, já prevista na seção 1.5; (d) seção 4 incorporada a partir da apresentação de arquitetura, pois a seção 3.2 a referencia.

Versão	Data	Alteração
2.0	17/10/2026	Fase 3. Árvore da seção 4.1 ampliada com infraestrutura (`ci.yml`, `Dockerfile`, `docker-compose.yml`, `.dockerignore`) e com os derivados `system_design_document.pdf` e `requirements.yaml`; novas seções 4.4 (pipeline de CI) e 4.5 (containerização); proteção de branch e tipos de commit `chore`, `ci` e `build` na seção 4.2. Sem mudança de comportamento do módulo: seções 1 e 2, `src/` e `tests/` inalterados.